rogii-wellbore-geology-prediction

6th Place - PF, PF, PF, PF, Physics and Row-Level Bagging

Rank	6
Team	k256.dev
Author	k256.dev
Topic ID	733226
Version	Original

Source: https://www.kaggle.com/competitions/rogii-wellbore-geology-prediction/discussion/733226
Retrieved with Kaggle CLI on 2026-08-06.

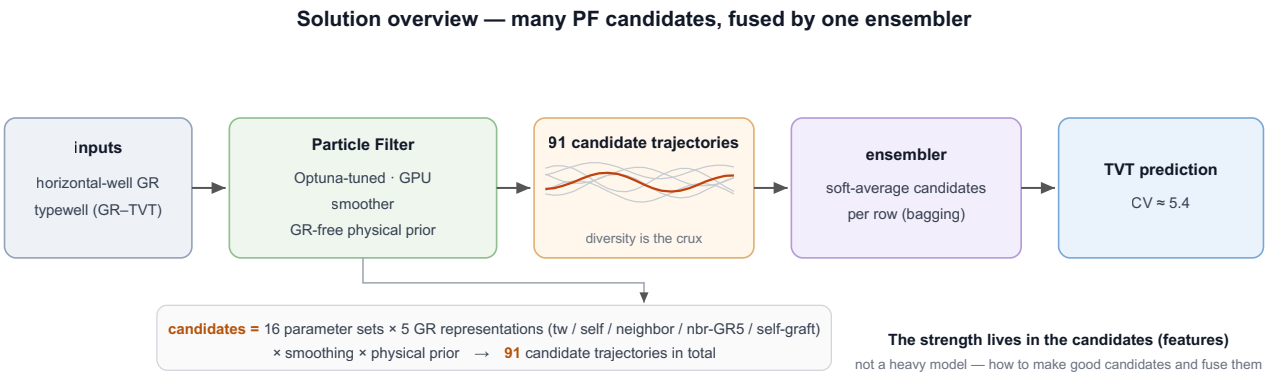
To the organizers — thank you for hosting such a wonderful competition. This is a largely unexplored problem, and although I am not well-versed in geology or time-series analysis, I found a great deal to learn. I finished **20th on Public and 6th on Private** — a large shake-up — and I also reflect on what caused such a big move in the standings. I present the solution following the trajectory of the accuracy improvements, so I hope you'll take a look. I'm also looking forward to reading the other participants' solutions.

1. Summary

The task is to take the gamma-ray log (GR) measured along a horizontal well, compare it against reference data provided separately, and predict how high in the rock column the well is passing (TVT).

My solution relied, from start to finish, on a **Particle Filter.** I ran many particle filters with different settings to produce many TVT estimates, and **fed a final set of 91 estimates into an NN model** to get the prediction. The mental picture is that **the NN fuses the 91 candidates row by row — it is doing "row-level bagging."** This reaches about CV 5.4. I am not a model specialist, so rather than elaborate on the model I fought this by relentlessly improving those 91 inputs (i.e., the features).

The final score was Private 5.984 / Public 5.626 / CV 5.4577.



---

2. Methods and results

2-0. The starting point

I joined 2 months before the deadline. Looking at the public notebooks, I saw the base was a **combination of a Particle Filter and a tabular model.** I also noticed these notebooks had a problem — they were **heavily overfit to the Public LB.**

A Particle Filter has a number of tunable parameters. This discussion confirmed that **a Particle Filter alone can reach fairly high accuracy**, and that **the public notebooks were under-tuned.** Tuning the parameters properly with Optuna was my first step. From there on, every improvement depended heavily on improving the Particle Filter.

2-1. Baseline — CV 7.8 / LB 7.168

The items I tuned with Optuna are in the table below. **At this stage there is nothing physical whatsoever.** Also, the **particle seed count is fixed at n_seed = 32.**

Parameter	Role
gr_sig (GR-likelihood σ)	tolerance of the GR match (how strict the alignment is)
gr_power	sharpness (exponent) of the GR likelihood
init_pos_std	spread of the initial TVT position
init_rate_std	spread of the initial slope (rate of change)
mom / vn / pn	slope momentum & transition noise (random-walk width of position/slope)
resamp	resample threshold (prevents particle degeneracy)
jump_prob / jump_std	probability of jumping to the other solution (branch), and its width
rate_clip / tvt_clip_margin	limits on slope/position drift

First I simply changed a **single** Optuna-tuned parameter and fed it the same way as the public notebooks. The result was CV 8.64 / LB 7.862 — overfit to the LB, but better than the public notebooks at the time.

Next I ran the experiment: **what if I concatenate the estimates from several Particle Filter parameter sets as input?** I tried a 3-input and a 5-input version (these two are different things), and both improved CV and LB.

Configuration	CV	Public	Δ(Pub−CV)	Private
1-input	8.64	7.862	−0.78	8.163
3-input	7.90	7.189	−0.71	7.729
5-input	7.80	7.168	−0.63	7.804

2-2. Hot-rodding the Particle Filter (GPU + a smoother) — CV 7.5 / LB 6.724

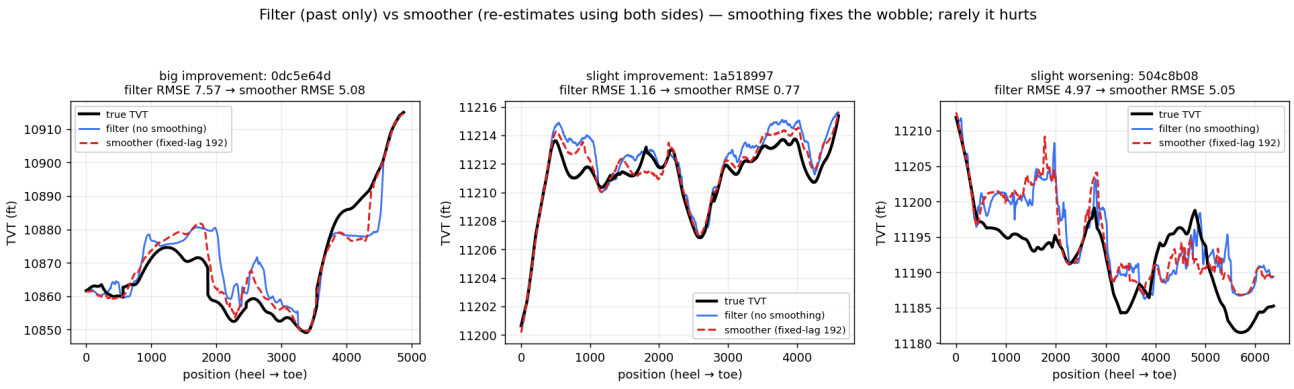
At the 2-1 stage, just running 3 PFs (seed = 32) already took about 4 hours to score, dominated by the Particle Filter. So I rebuilt the **Particle Filter to run on GPU.** This made experiments and inference **~200× faster,** letting me run far more trials.

A by-product of this rebuild was that I could add a **smoother mechanism** to the PF. An ordinary particle filter uses "only the past" to estimate the position at each step (causal). A smoother instead **keeps the particles' ancestry (which particle came from where) and re-estimates the past after seeing a little of the future GR.** Because it uses GR context on both sides, the estimate is more stable, and places that momentarily got lost

in noise get corrected after the fact. What I use here is a **fixed-lag particle smoother**, following Kitagawa (1996) / Doucet & Johansen (2009) (see References).

Feeding this into the tabular model, a smoothing width of **smooth = 192 gave CV −0.3 / LB −0.4.** However, **the longer the lag, the heavier the compute** — each particle's ancestry must be kept over a long window — so it runs slower than the plain (no-smoothing) PF. Measured on an RTX 5070, one pass (one parameter set × the tw representation) over all 773 wells takes **~4.5 min for plain PF → ~4.9 min for fixed-lag 16 (+~10%) → ~13 min for fixed-lag 192 (~2.9× plain).** A short lag is nearly free, but stretching it to 192 becomes clearly heavy (and is impractical without the GPU port).

Configuration	CV	Public	Δ(Pub−CV)	Private
5-input	7.80	7.168	−0.63	7.804
5-input + smooth	7.50	6.724	−0.78	7.404



Above: pf_1 (tw representation) run with no smoothing (filter) and with fixed-lag 192 (smoother), with three cases picked by the per-well RMSE difference. **Left = big improvement:** where the filter wobbles and overshoots early on, the smoother — using GR context from both sides — pulls it back (7.57 → 5.08). **Middle = slight improvement:** on an already-easy well the effect is small but consistent (1.16 → 0.77). **Right = slight worsening:** very rarely, smoothing nudges it slightly worse (4.97 → 5.05). On balance, the fix-the-wobble effect dominates.

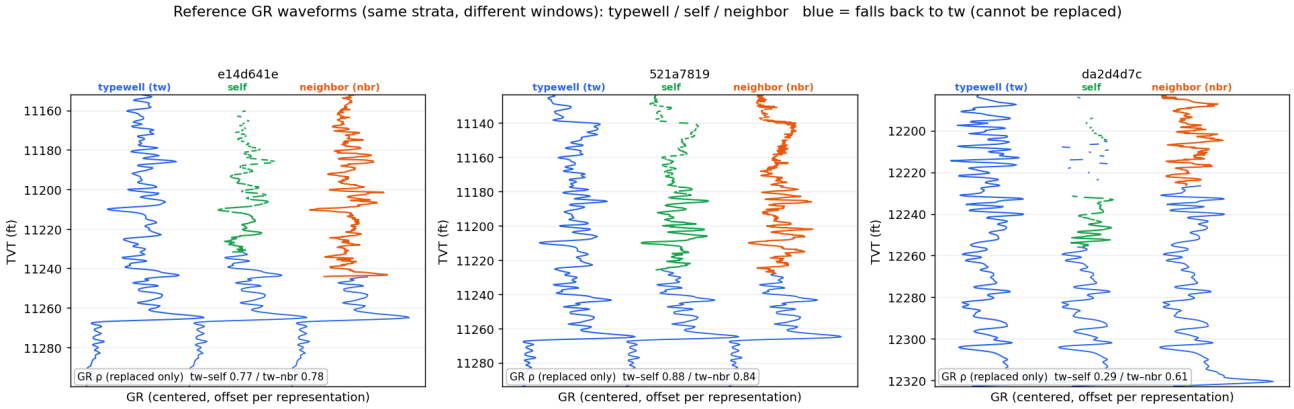
2-3. Alternative GR representations (self / neighbor) — CV 6.98 / LB 6.026

From this discussion I learned that the typewell GR (henceforth **twGR**) is noisy, and that there is a notion of a "**pseudo-typewell.**"

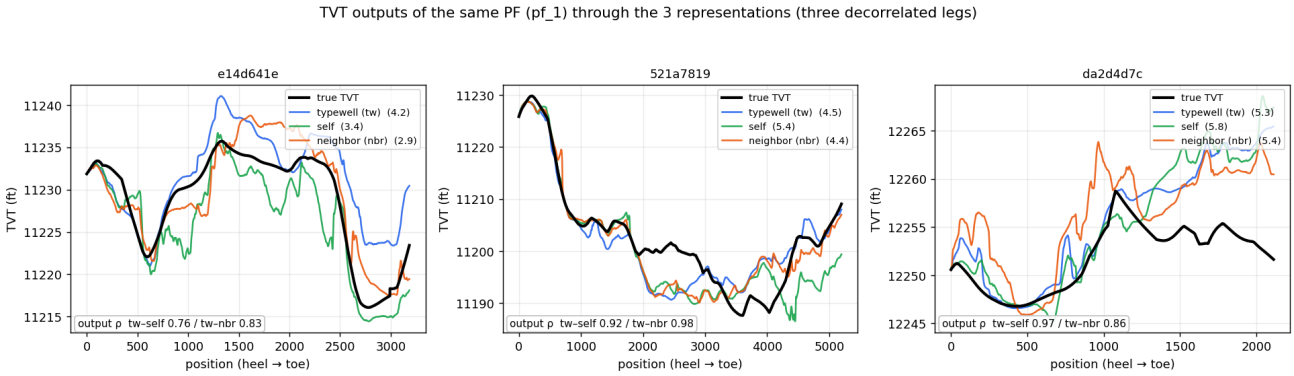
Thinking about how to apply this to our task, I noticed that **neighboring train wells are provided** (I call this **neighbor**). By overwriting the twGR with the known GR of a train neighbor within 1500 ft, you get a pseudo typewell GR (**nbrGR**).

There is a second idea of the same kind — **self.** It uses the well's own prefix (the known interval on the heel side). Since TVT and GR are both known there, I overwrite in the same way (**selfGR**).

The estimates inferred from these tvt–GR pairs are **strongly complementary** to the twGR estimate. The correlations are roughly selfGR ≈ 0.79 and nbrGR ≈ 0.72 against twGR — accurate on their own, yet sufficiently decorrelated. Different references fail in different ways, so their independent errors cancel downstream.



Above are the GR waveforms of the same well seen through three references — twGR / selfGR / nbrGR (each centered and offset side by side). **The large-scale shape is shared (the same strata), but the fine detail differs slightly.** This "same strata through a different window" difference is the source of the decorrelation.



And when the **same PF (pf_1) is run through the three representations,** the TVT outputs become three mutually decorrelated legs (per-well RMSE in the legend). Each tracks the true TVT on its own, but since they miss in different directions, their errors cancel in the downstream ensembler.

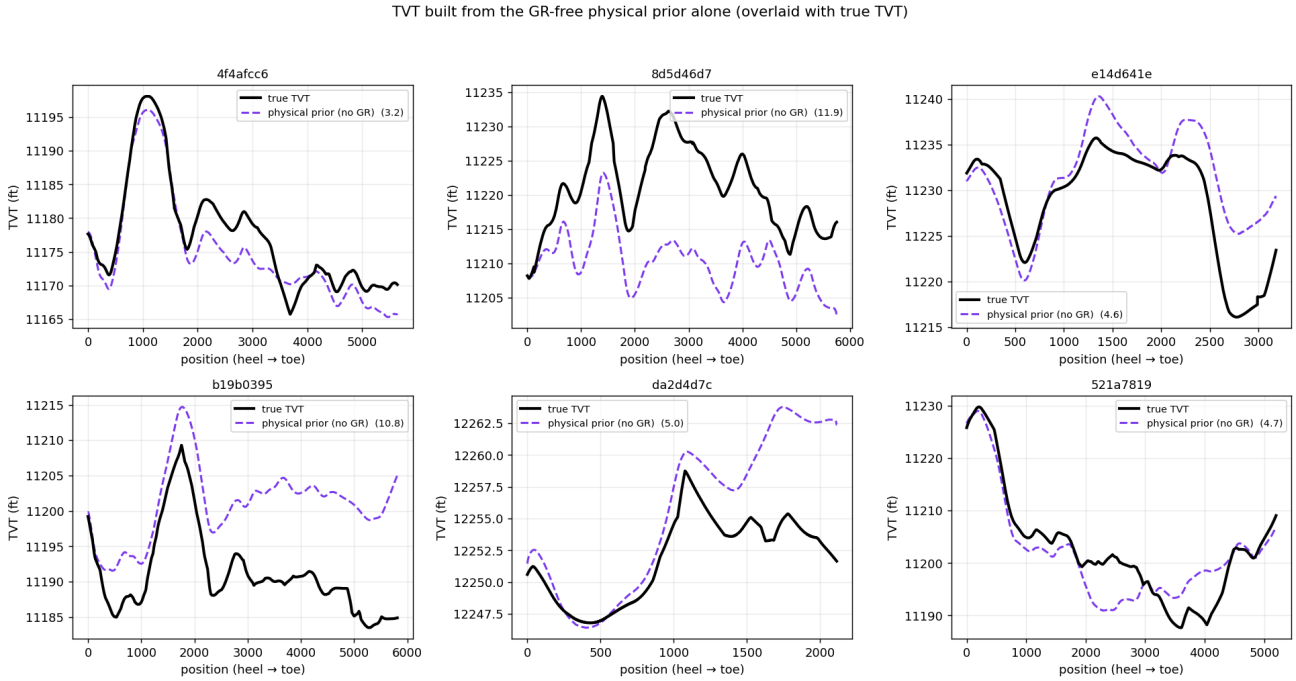
Generating these two and expanding to 15 inputs to the tabular model gave **CV and LB both −0.50.**

Configuration	CV	Public	Δ(Pub−CV)	Private
2-2 (5-input + smooth)	7.50	6.724	−0.78	7.404
2-2 + selfGR + nbrGR	6.98	6.026	−0.95	7.112

2-4. Injecting a physical constraint — CV 6.50 / LB 6.43

At this point CV (~7.0) and Public LB (6.0) diverged badly. Following the discussion argument to "**combine physical constraints,**" I built a **prior over TVT from the well geometry (X, Y, Z) and the known TVT alone, using no GR whatsoever.**

The idea: even without GR, if you know how far and in which direction a well went (geometry) and how the surrounding wells are tilting (the dip of the strata), you can predict how TVT will change. I split each well into segments, estimate the local dip in each, share it spatially, and integrate the dip to reconstruct the TVT trajectory. Crucially, **this must be built in a strict out-of-fold manner**: mixing any information from the evaluation wells into the dip field or the calibration makes only the CV look unfairly good, and it then collapses on Private (the seed of a shake-down).



Above are **TVT waveforms built from the physical prior alone, using no GR at all,** overlaid with the true TVT (per-well RMSE in the legend). **Even without relying on GR, geometry and the neighbors' dip alone already track the large-scale TVT undulation fairly well** (the wells it fits land at 3–5 ft, the ones it misses exceed 10 ft — there is spread).

This physical prior is **weak on its own (CV 10.1 / LB 10.8),** but it shows its worth when **injected into the PF likelihood.** Adding Optuna-tuned parameters to 2-2 under the assumption of this injection gave **CV −0.5 / LB +0.5** — and here, **for the first time, CV and LB agreed.**

Configuration	CV	Public	Δ(Pub−CV)	Private
physical constraint only	10.1	10.8	+0.70	9.342
twGR-prior PF alone	7.8	7.88	+0.08	7.78
2-3 (selfGR + nbrGR)	6.98	6.026	−0.95	7.112
2-3 + prior PF	6.50	6.43	−0.07	6.67

Indeed, this configuration is confirmed to **transfer correctly to Private as well** (CV 6.50 / Public 6.43 / Private 6.67). Even if the Public LB had looked worse here, this shows that **choosing to trust the CV and press on was the right call.**

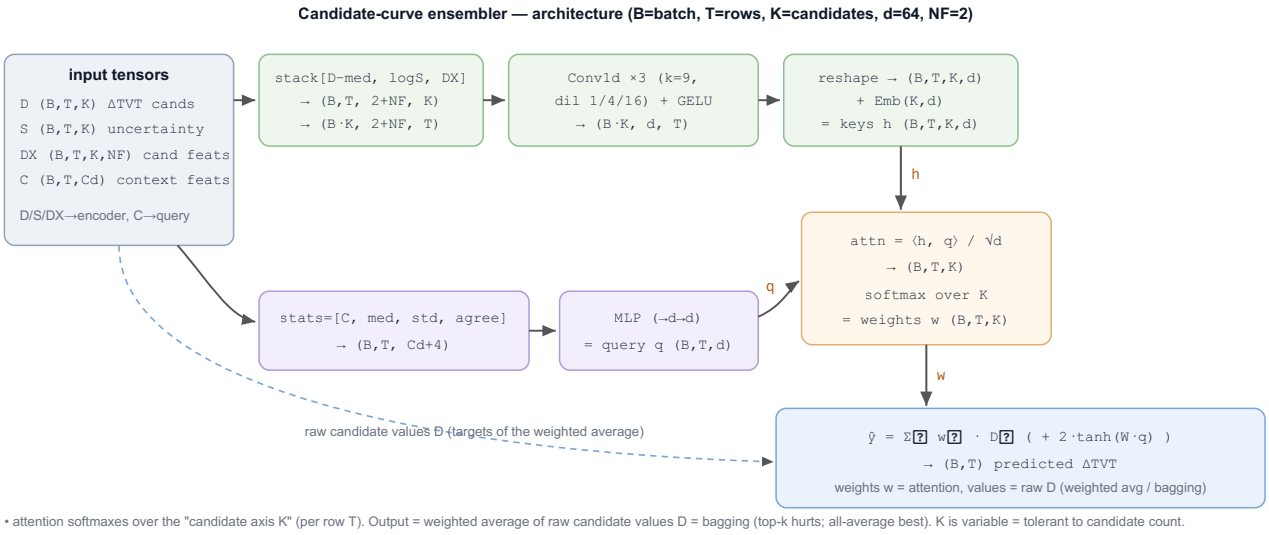
This "CV and LB agreed" was the biggest turning point of the competition.

2-5. Fusing many leftover PFs with an input-tolerant ensembler — CV 5.94 / LB 6.17

I had a large pile of PFs that I had tuned with Optuna but that **had not, on their own, led to any improvement.** At some point, instead of discarding them, I **fed them all in together — and the accuracy jumped, dropping CV below 6.0 (CV 5.94 / Public 6.17).**

Why did it work? First, dissecting the error, the residual against a strong PF had self-correlation of almost 1.000 — meaning **the independent per-row signal is basically exhausted, and the error lives in a per-segment choice of "which solution do we sit on."** The problem was not row regression but, per segment, **choosing which of many candidates to sit on.**

So I built a **candidate-curve ensembler** that **lines up each PF × representation TVT trajectory as "one candidate curve," and lets a lightweight attention soft-average the candidates per row.** The tabular model could only concatenate a fixed-length set — **limited in expressive power** — whereas the ensembler can line up **any number of candidates.** That was decisive: it let me throw in **all the PFs I had lying around that hadn't helped on their own,** and that is what suddenly worked.



The candidates I fed in were **6 PFs (pf_1 / pf_2 / pf_3 / r0_seed32 / r1_seed32 / pfA) × 5 representations = 30.** The five representations differ in **which template the GR is matched against**:

- **typewell**: the given vertical-well template (2-3).
- **self**: the well's own known prefix GR as a pseudo-template (2-3).
- **neighbor**: a spatially near (XY) train well's GR as a pseudo-template (2-3).
- **nbr_gr5**: neighbors chosen not by XY distance but by **GR similarity — the top-5 train wells whose GR (stratigraphy) matches**.
- **self_graft**: extends "self" beyond its known range by **extrapolating (grafting)** with a tw→self calibration, widening the range "self" can cover.

Of the 6 PFs, only one (pfA) carried the physical (GR-free anchor) constraint.

The key design finding is that **it acts as a broad soft average (bagging), not a one-of-N selector.** Indeed, restricting to the top-k hurt, and softmax over all candidates was optimal. Since you cannot tell "which

solution is correct" from GR alone, the right move is **not to force a choice, but to average probabilistically and hedge.**

At this stage the final prediction was a **fixed-weight 3-leg blend of GBM (tabular) + a sequence net (TCN) + the ensembler.** The standalone CV and blend weight of each leg:

Leg	standalone CV	blend weight
GBM (tabular)	6.65	0.05
sequence net (TCN)	6.32	0.51
candidate-curve ensembler	6.19	0.44
3-leg blend	**5.94**	—

Configuration	Candidates	CV	Public	Δ(Pub−CV)	Private
2-4 (Tabular, 15 candidates)	15	6.50	6.43	−0.07	6.67
feed the whole pile of leftover PFs into the ensembler	30	5.94	6.17	+0.23	6.382

2-6. Stacking one leg at a time — toward Submission A / Submission B (the final form)

From CV 5.94 onward, each single addition — **more decorrelated candidates, or stronger smoothing** — steadily lowered both CV and Public. In the order I added them:

Addition	Candidates	CV	Public	Δ(Pub−CV)	Private
(reached in 2-5)	30	5.94	6.17	+0.23	6.382
① feature-combine, retrain one model	46	5.75	5.707	−0.04	6.166
② fixed-lag → full smoothing	46	5.70	5.652	−0.05	6.097
③ +5 decorrelated PFs (**Submission A**)	71	5.587	**5.458**	−0.13	5.997
④ add dec10 / tw-affine (**Submission B**)	91	5.458	5.626	+0.17	**5.984**

① **Feature-combine > ensemble.** Re-training a single model on combined features (candidates) beat blending separately-trained models by ~2.6×. Since the ensembler's strength is "candidate quality × diversity," combining beats mixing.

② **Full smoothing.** I replaced the fixed-lag smoother of 2-2 with a whole-interval one (an ancestral smoother with the lag extended to the full sequence length — the same lineage of Kitagawa 1996 / Doucet & Johansen 2009; the whole-interval version is prone to path degeneracy). The ensembler directly enjoyed the better candidates, and **CV −0.043 transferred fully to Public.**

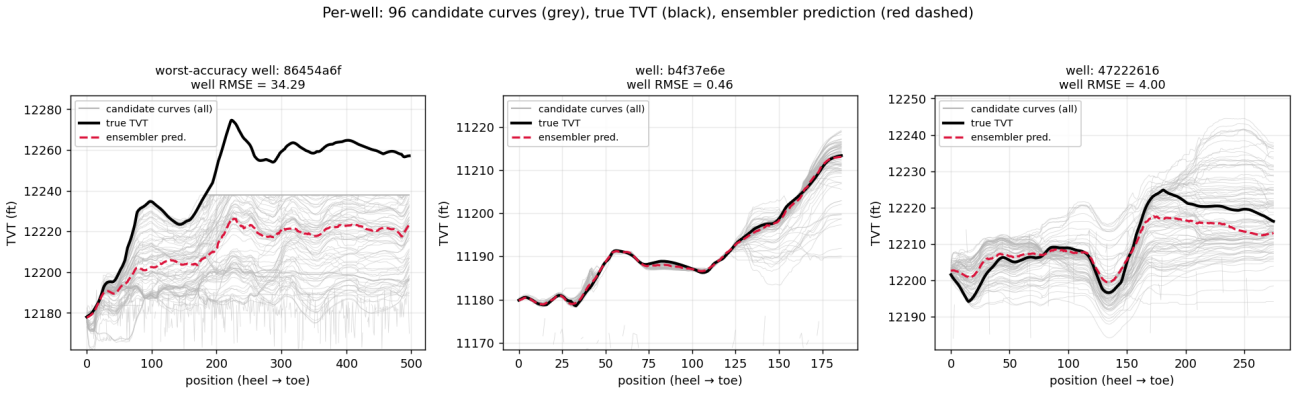
③ **Stacking decorrelated PFs (Submission A).** I added five separately-tuned PFs with no anchor (physical constraint) and no emission, growing candidates 46 → 71. **CV 5.70 → 5.587, Public 5.637 → 5.458 — an even-better-than-CV transfer (−0.18).** This is the main lever, and the key is: **only candidates whose way of being wrong points in a different direction help, even if individually weak.** Adding candidates 0.99-correlated with what I had did nothing.

④ **dec10 / tw-affine (Submission B).** I further added dec10 (a PF on MD-decimated, noise-averaged GR) and tw-affine (a PF with prefix-calibrated GR), bringing candidates to 91: **CV 5.4577 / Public 5.626.**

Post-processing (de-shrink). For both Submission A and B, the final touch applied to the ensembler's blended prediction is a **one-parameter calibration, a global gain a≈1.03.** Because the ensembler is a weighted (soft) average of candidates, the prediction's amplitude ends up slightly shrunk relative to the truth; the global gain restores it. It is a small add of **~−0.015 CV,** but a **one-parameter global correction is hard to overfit and transfers robustly** — in contrast to well-level constant-shift post-processing, which back-fired on the LB (§ 3-2, § 5).

Conversely, making the ensembler itself stronger (cross-attention, injecting candidate-to-candidate relations, injecting external legs as candidates) **all saturated at ~0.99 correlation with the base** — a single-model gain that never made it into the ensemble. The headroom was, in the end, only ever "decorrelated candidates from a different information source."

Finally, here are the final candidates visualized for three wells (grey = all candidate curves, black = true TVT, red dashed = ensembler prediction).



Left = the worst-accuracy well: the true TVT sits above every candidate — all candidates got the datum branch wrong. Since the ensembler is a weighted average of candidates, on a hard well where the candidates fail to bracket the truth it **cannot possibly reach it** (this is exactly the datum wall of 2-4 / § 3-2). **Middle / right = two random wells:** when the candidates bracket the truth, the ensembler hits it well.

---

3. Discussion

3-1. The biggest driver of the accuracy gains

I believe the single biggest driver was that **the PF outputs themselves carried a lot of diversity.**

Concretely: **trajectories with the physical prior (the GR-free constraint) and without it,** matching against **a typewell or against a nearby well instead,** trajectories derived from the **prefix (the known interval),** plus the **overall lift from full smoothing** — I ran **everything I could think of** through the PF. The result was **a lot of diversity from the PF alone.** Each individual candidate is not strong at all — around **10.0–7.8 in overall RMSE** on its own.

Bluntly, what I am doing is **bagging a large number of PF outputs.** But it turned out to be a nice example of how **simply lining up and fusing candidates that are wrong in different ways can push the accuracy this far.**

3-2. The bias of the Public LB, and CV / Public / Private transfer

I once said in the discussions that "**the Public overfits under a certain condition.**" My belief is that **methods that depend only on the GR match overfit to the Public.**

The table below shows results with and without the physical constraint. **GR-only methods (no physics) transfer strongly and positively from CV to Public** (Public looks much better than CV). Once the physical constraint is added, **both Public and Private line up with CV** (the transfer becomes honest).

Configuration	CV	Public	Δ(Pub−CV)	Private	Transfer
5-input + smooth (GR-only)	7.50	6.724	−0.78	7.404	Public much better = GR overfit
self + nbr (GR-only)	6.98	6.026	−0.95	7.112	same (CV ≫ Public)
+ physical prior	6.50	6.43	−0.07	6.67	**CV ≈ Public ≈ Private**

And this was **confirmed by the final Private results.** Ordering the two finalists — **CV: Submission B < Submission A** (B better), **Public: Submission A < Submission B** (reversed), and **Private: Submission B (5.984) < Submission A (5.997), agreeing with CV.** So CV predicted the right direction; **the Public LB, which made Submission A look better, was the single outlier.** The configurations I added in Submission B only looked overfit on Public; on the more representative, harder Private set, they were, as CV said, slightly better.

3-3. How to build a physical constraint that risks a shake-down

The physical constraint of 2-4 is very powerful, but **built wrongly it easily overfits the Public/CV and collapses hard on Private (a shake-down).** This is the single most important trick.

The constraint builds a "dip field of the wells" and its "calibration coefficients" from other wells' information. If you let **any information from the well being evaluated (validation) leak into building the field or the**

calibration, you have peeked at the answer, and CV becomes unfairly good. Then you can no longer observe CV–LB transfer correctly, and a submission chosen by trusting the Public order collapses on Private.

The fix: **always build it in a strict 5-fold (out-of-fold) manner.** That is, construct the dip field, the calibration coefficients, and the neighbor pool from **only the training-side wells of that fold,** fully excluding the evaluation wells, and rebuild the field and calibration per fold. Since my goal was to grow CV into a trustworthy metric, this rigor was decisive. Whenever you inject a "learned prior" like a physical constraint, treat leakage as **the seed of a shake** and always build it with the evaluation targets excluded.

3-4. Why per-row regression (tabular alone) does not work

Early on, the tabular model (GBDT) led, but as accuracy improved it was fully replaced by a sequence model + candidate selection. The reason is fundamental.

In one line: **it is not a problem you can solve one row at a time.** Against a strong PF output, the remaining error is no longer "small per-row wiggles"; it is that **a whole segment is shifted, as a block, one way or the other.** Because of the periodicity of the strata, GR looks almost identical at two places, above and below — so telling "is this segment the upper one or the lower one?" from GR is essentially a coin flip (you can't).

A per-row feature vector carries no information about "which of the two solutions this whole segment sits on." So the tabular model plateaus no matter how many features you add. In contrast, lining up the PF's "single coherent trajectories" as candidates and soft-averaging them naturally expresses this "whole-segment choice." In short, **tabular is not bad — per-row regression was the wrong frame for the shape of the problem (a whole-segment two-way choice).**

3-5. Designing the CV so you don't shake down on Private

The thing I valued most for avoiding a shake-down was **keeping the variance between folds small.**

The reason is that **Train / Public / Private are most likely split completely at random,** and **each is small — on the order of 150 wells.** With a small random split, there is always a chance that **the hard wells happen to cluster into one split and you draw a downswing.** Whether Public or Private turns out to be a "bad draw" is not something you can control.

That is exactly why **a design whose score never collapses on any fold or any split — i.e., one with no downswings — pays off.** A model with low inter-fold variance won't miss badly whatever random 150-well draw Public / Private happens to be. Conversely, a model with the same average performance but high inter-fold variance always carries the risk of hitting a downswing somewhere and shaking down. **Removing the downswings matters as much as raising the average** — that is my conclusion.

Measured on the final submission, **each of the 5 folds (≈155 wells each) lands roughly in the 4–6.7 range, and the overall CV averaged over the 5 folds barely moves — 5.40–5.43 (std ≈ 0.015) no matter how the split is re-drawn.** In the early baseline (2-1) the per-fold RMSE swung from about 6 to 10, so **the inter-fold spread has shrunk substantially.** I believe this "resistance to downswings" is one reason it did not collapse on Private either.

3-6. Why the Optuna-tuned PF parameters didn't overfit the CV

I tuned the PF parameters by **randomly selecting 600 wells (or 150 wells) from the 773.** The natural worry is: "wouldn't that overfit to the wells you happened to pick?"

First, **the PF is a deterministic method with not that many parameters,** so it is inherently resistant to overfitting. That said, **when I optimized on few wells, I did confirm that overfitting occurs.**

However, that overfitting appears to be **only against twGR (the representation that matches against the typewell's GR).** I did the tuning on twGR and then **reused those tuned parameters as-is for the other representations — nbrGR, selfGR, and so on (2-3).** I believe that **applying parameters overfit to twGR onto representations of a different nature is what diluted the overfitting.**

3-7. Why the shake-up happened

The large turnover inside the gold zone was **not** caused by overfitting to the Public. It happened because **there existed methods that were excellent for CV and Public but underwhelming on Private.** The very methods of those who got shaken down are, in my view, the valuable ones. **This is not the same as the public-notebook overfitting — please don't lump them together.**

The final standings saw a large shake-up, and much of the Public top turned over. Looking at the solutions that survived at the top, **teams that leaned on physics-based models — HMMs, particle filters — climbed almost across the board.** I can't say for certain whether this was mere split noise or the result of deliberately choosing a different domain (i.e., a more physics-oriented formulation), but at the very least the **dominant solution domain clearly differed between the Public top group and the Private top group.**

After the competition I had a brief chance to talk with another team, and they raised the point that **"approaches that don't rely on a physical model may be prone to CV overfitting."**

Seen that way, **that I ended up on the shake-up side (the one that rose) owed a good deal to luck.** My read is that the Public-top solutions that did not strongly depend on a physical model — however excellent as methods — **could not escape the accuracy drop from domain shift on Private.**

---

4. What worked

The methods that were the reasons for improvement:

- **Optuna-tuning the Particle Filter's parameters** (GR-likelihood σ, transition noise, resample, jump, …) — the foundation for good candidates.
- **Concatenating estimates from several PF parameter sets** as input (3-input, 5-input — stepwise gains).
- **Porting the PF to GPU** (~60× faster) — enabled far more trials and the smoother.
- **The smoother mechanism.** Stabilizes the estimate with two-sided GR context; lag = 192 gave CV −0.3 / LB −0.4, leak-free and transferring cleanly.
- **Adding several alternative GR representations.** Beyond twGR: self, neighbor (by XY distance), GR-similarity neighbor, and a self-graft (self extrapolated via calibration) — five in all. Complementary candidates, decorrelated (≈0.7–0.8) from twGR yet accurate.
- **Injecting a no-GR physical constraint into the PF likelihood** (always built 5-fold). Aligned CV and LB and neutralized the Public leniency of GR-over-reliance.
- **An ensembler that soft-averages candidates as "one curve each"** (a hedge, not a selection).
- **Stacking decorrelated configurations** (only candidates that are wrong in a different direction help) — the sole headroom late in the game.
- **Feature-combine > ensemble** (combine and retrain, rather than train separately and blend).

- **Full smoothing** (whole-interval) — the ensembler transfers the candidate-quality gain fully.
- **Sequence-NN (TCN) residual blend** (largest lift on weak backbones).
- **De-shrink post-processing** (restore the averaged, shrunk prediction with a single global gain $a \approx 1.03$) — **CV 5.587 → 5.572 (−0.015)**. A one-parameter global correction, so it is physically motivated and transfers robustly.
- **FFBS-type particle smoothing (Godsill et al. 2004).** As a whole-interval ancestral smoother it **did work** (raises candidate quality). But it is **too slow, so I did not adopt it in the final pipeline** — I replaced it with the lighter fixed-lag / full smoothing.
- **"Keep the PF plain and single; put the cleverness downstream" / match the inference seed to the tuning** (avoid overfitting).

---

5. What didn't work

Nearly everything I tried that failed or back-fired:

- **Tuning PF parameters per well-category** → overfits; loses badly to a single fixed PF.
- **Naive averaging of PF outputs** → averaging estimates committed to different solutions collapses to the mid-point and breaks.
- **selfGR as an output blend (a "prediction")** → double-counts with existing post-processing; non-transferring and harmful (useful as a feature).
- **Strengthening the physical (spatial) constraint standalone** → lowers CV but back-fires on LB (the higher the prior density, the worse the transfer).
- **Well-level constant-shift post-processing (vote-hedge, …)** → OOF −0.075 becomes LB +0.184.
- **Sharpening the ensembler's selection (top-k / cross-attention / candidate-relation / external-leg injection)** → all saturate at ~0.99 correlation with the base.
- **Trying to solve it with tabular (row regression) alone** → the frame doesn't fit a whole-segment two-way choice.
- **A PF-free 2D image CNN (GR-match image → U-Net)** → good single CV, but LB collapses to 9.46.
- **Baking the physical constraint into tabular features** → kills the good GR-derived transfer (the two trade off).
- **Attacking the two-way choice directly (global-MAP / Viterbi / sparse mode-switching)** → GR can't pick the branch; what "worked" was just the physical constraint underneath.
- **Conditionally pulling the prediction back to a known value** → none of the criteria for which to pull ever hit.
- **Learning the blend weights (meta-learning)** → loses to hand-set fixed weights.
- **Heavy sequence architectures (iTransformer / Mamba-family / deep TCN / BiGRU)** → saturate re-processing the same candidates; poor speed/benefit.
- **Using neighbor distance as a proxy for "stratigraphic closeness"** → distance ≠ stratigraphy (the best GR reference is ~5,800 ft away on median).
- **Backward data augmentation (a reversed-well synthetic task)** → in a weak engine (early on) CV −0.05 transferred well to LB −0.16, but in the final strong PF/ensembler it no longer helped — it actually caused a **+0.2 worsening (a reverse transfer).** The stronger the engine, the more the DA gain decays or flips.

- **Pre-training on synthetic wells** → it didn't work. But this was **probably because my generation (how the synthetic data was made) was poor** — I can't conclude the approach itself has no promise.

---

6. Additional notes

- **Environment & compute cost**:
  - **Training / experiments (local):** GPU = RTX 5070 / CPU = AMD Ryzen 7 7700 / RAM = 32GB DDR5. All PF/NN training and every experiment ran on this single machine (every measured timing in this write-up is from this environment).
  - **Submission (Kaggle notebook):** GPU = T4 × 2. Since the GPU generation, VRAM, and numerical behavior differ from local, the submission side had its own pitfalls to watch for — OOM on the hidden test, bf16 math fallback, and the like.
- **The PF parameters used in the final submission (Submission B)**: Submission B's candidate pool is **16 Optuna-tuned PF parameter sets,** each run through 5 GR representations (tw / self / neighbor(XY) / GR-similarity neighbor / self-graft) and several lags, for **91** candidate trajectories in total (including 15 columns where tw is re-calibrated with a prefix affine, and 5 MD-decimated columns). The key knobs of the 16 sets are below. Common settings: **n_seed = 32, whole-interval smoothing, grid step 0.2 ft.** Column meanings: σ_def = base width of the GR likelihood; vn/pn = process noise (rate / position); resamp = resampling threshold; anchor = strength (mult) of the GR-free physical constraint; emis = emission temperature w_nn of the learned similarity.

bank	parts	gr_power	σ_def	vn	pn	resamp	jump	anchor	emis	role
pf_1	600	2.19	114	.00014	.00068	.061	.002	–	.02	original Optuna group (per-set GR-σ / process-noise tuning)
pf_2	600	2.00	71	.00011	.01113	.308	–	–	.02	″ (narrow σ, high resamp)
pf_3	600	2.07	67	.00007	.04339	.055	.002	–	.02	″ (large pn = high position noise)
r0	400	1.78	119	.00006	.00050	.032	.002	–	.02	re-tuned under a different seed
r1	400	1.46	83	.00012	.00028	.037	.002	–	.02	″ (loose likelihood gr_power)
pfA	600	1.85	87	.00009	.00061	.186	.002	**√20**	.01	**physical anchor (mult 20) + emission** (the pillar that aligns CV=LB)

bank	parts	gr_power	σ_def	vn	pn	resamp	jump	anchor	emis	role
pf_1'	800	2.99	131	.0009	.0405	.053	–	–	.02	a separately-tuned re-tune group (sharp likelihood, gr_power≈3)
pf_2'	500	2.08	95	.0010	.0366	.064	.002	**✓20**	.02	〃 (with physical anchor)
pf_3'	600	1.49	125	.0017	.0028	.212	–	–	.02	〃 (loose likelihood, high resamp)
pf_4'	700	2.31	92	.0020	.0107	.040	–	**✓20**	.02	〃 (with physical anchor)
p_1	700	2.08	103	.0013	.0078	.046	.002	–	**0**	**5 decorrelated PFs with no anchor and no emission** (Submission A's main lever ③). Turning emission off preserves candidate diversity, and the ensembler gained the most
p_2	600	2.01	90	.0009	.0015	.207	.002	–	**0**	〃
p_3	800	2.36	122	.0016	.0086	.220	.002	–	**0**	〃
p_4	800	2.23	110	.0013	.0044	.140	–	–	**0**	〃
p_5	700	2.44	88	.0013	.0063	.030	–	–	**0**	〃
dec10	600	2.19	114	.0014	.0068	.061	.002	–	.02	pf_1's parameters applied to **MD ×10-decimated** GR (a noise-averaged, decorrelated leg, Submission B's ④)

All values are the actual Optuna-found settings (for the exact full hyperparameters, see `pf_banks_config.json` in the released code). The point is **to scatter the direction of the errors, not single-model performance** — which is why σ, gr_power, resample, and the presence/absence of anchor/emission are deliberately varied.

- **Code**: I will share the code on GitHub later.
- **Submittion Notebook**: https://www.kaggle.com/code/k256net/public20th-private6th-particle-filters

(Sorry, it's not well organized, so it might be a bit hard to read. Please take it as a rough reference.)

---

7. Closing

My approach is **probably quite different from the other top finishers'.** I don't have deep model knowledge. Instead, I got here by **relentlessly improving the features that the Particle Filter produces.** Rather than bringing in a powerful model, I focused on "how to make good candidates and how to fuse them" — a somewhat unusual solution in that sense.

My thanks to the organizers and Kaggle, and to everyone who shared insight in the discussions. It was a genuinely novel and fascinating competition. Thank you.

If you have any questions, feel free to drop a comment — I'll answer as best I can.

(This write-up was written in collaboration with Claude, so there may be a few missing or incorrect points.)

References

- N. J. Gordon, D. J. Salmond, A. F. M. Smith (1993). Novel approach to nonlinear/non-Gaussian Bayesian state estimation. IEE Proceedings F, 140(2), 107–113. — the original bootstrap (SIR) particle filter.
- G. Kitagawa (1996). Monte Carlo Filter and Smoother for Non-Gaussian Nonlinear State Space Models. Journal of Computational and Graphical Statistics, 5(1), 1–25. — particle filter + (fixed-lag / whole-interval) particle smoother; the basis of the smoother in this write-up.
- A. Doucet, A. M. Johansen (2009). A Tutorial on Particle Filtering and Smoothing: Fifteen Years Later. — the standard survey of particle smoothing (incl. fixed-lag and ancestral smoothing, and path degeneracy).
- S. J. Godsill, A. Doucet, M. West (2004). Monte Carlo Smoothing for Nonlinear Time Series. JASA, 99(465), 156–168. — FFBS (mentioned under "What didn't work").